

The Distributed Dead Man's Switch

CMPE 273 (San José State University) — 2026-04-30

Author: yashashav.dk@gmail.com

Abstract

This paper covers two questions in failure detection for distributed systems: should heartbeats be pushed by workers or pulled by a monitor, and how does the monitor decide when a "missing" node is officially "dead"? We compare push and pull empirically on a single-monitor / N-worker testbed implemented in Go over gRPC, then propose and implement a Phi Accrual Failure Detector (Akka variant) and compare it against a Fixed Window baseline. Source code, benchmarks, and the data behind every figure are reproducible from the included repository.

1. Introduction

The dead man's switch is a classical failure-detection primitive: a worker periodically signals its liveness to a monitor, and the monitor takes action when the signal stops. This is the building block under leader leases, distributed cache eviction, autoscaling, and most cluster-membership protocols. Designs differ on two axes: who initiates the heartbeat (push or pull), and how absence of heartbeat is interpreted (a fixed timeout, or an adaptive estimate). We treat both questions in turn.

The implementation is a single Monitor process observing N Worker processes over gRPC. A `--mode` flag selects push or pull; a `--detector` flag selects Fixed Window or Phi Accrual. Workers can inject lag, drop heartbeats, or self-terminate to produce reproducible failure scenarios.

2. Heartbeat Strategy

2.1 Push model

In the push model, each Worker opens a long-lived stream to the Monitor and sends a small heartbeat record every `hb_interval`. The Monitor's only role is to read the stream and timestamp arrivals using its own clock.

Network cost is $O(N \cdot 1/\text{hb_interval})$ heartbeats per second; with $N=1000$ and `hb_interval=1s`, that is 1000 messages/s into one process. Connection count is $O(N)$ but each connection is a long-lived TCP stream, not a per-heartbeat handshake. The Worker initiates the connection, which means push is firewall-friendly: only the Monitor's port has to be reachable, not every Worker's.

2.2 Pull model

In the pull model, the Monitor maintains a per-Worker goroutine that calls `Ping` on the Worker every `poll_interval` and treats successful replies as heartbeats. Network cost is

symmetric: $O(N \cdot 1/\text{poll_interval})$ RPCs per second. Connection count is also $O(N)$ and connections can be reused across calls.

Pull's structural drawbacks are two: every Worker must be reachable from the Monitor (hostile to NAT, asymmetric firewalls, or per-Worker auth gateways); and the Monitor's load grows directly with N , with no opportunity for the Worker to throttle itself when it knows it's healthy.

2.3 At 1000 workers (empirical)

Our benchmark sweeps $N \in \{10, 100, 1000\}$ for both modes and reports Monitor peak RSS, cumulative CPU, and detection latency. (Run `scripts/bench_push_pull.sh`; figure `paper/figures/rss.png` is generated by `scripts/plot.py`.)

[image]

[image]

The headline finding is that, at $N = 1000$, push consumes $\sim 3 \times$ less Monitor RSS than pull because pull holds an outbound stub-and-conn per worker, while push reuses one inbound stream. Per-tick wire bytes are similar in both directions; the differences are in per-connection accounting and per-poll context allocation on the Monitor.

2.4 Behind a firewall

Push wins: the Worker is the connection initiator, so only the Monitor's port has to be open to the network the Worker lives in. Pull requires the Monitor to dial the Worker, which means the Worker must be on a routable address (or the operator must run a tunnel). This is the dominant reason production heartbeat systems (Cassandra, Consul, Kubernetes kubelet $\rightarrow$ control-plane) prefer push.

3. The Timeout Dilemma

3.1 Fixed Window

The simplest detector picks two multipliers `k_miss` and `k_dead` and declares MISSING after `k_miss * hb_interval` of silence and DEAD after `k_dead * hb_interval`. With the standard defaults of `k_miss=3`, `k_dead=10`, that is 3 seconds and 10 seconds at `hb_interval=1s`. The detector is deterministic and trivial to reason about — and sensitive to jitter, because a single delayed heartbeat past `T_missing` is indistinguishable from a real failure.

3.2 Phi Accrual

Hayashibara, Defago, Yared and Katayama proposed the Phi Accrual Failure Detector in 2004. The idea: rather than a hard threshold, accrue a *suspicion* value `phi(t)` that represents the negative log-probability that the worker is still alive given the empirical distribution of past inter-arrival times.

Concretely: maintain a sliding window of the last N inter-arrival samples. Compute mean μ and standard deviation σ . Then:

```
elapsed          = now - last_heartbeat
```

$$P_later(t) = 1 - F(t) \quad \text{where } F \text{ is the CDF of Normal}(\mu, \sigma)$$

$$\phi(t) = -\log_{10}(P_later(t))$$

Akka's production code uses a polynomial approximation to avoid `erfc`:

$$y = (\text{elapsed} - \mu) / \sigma$$

$$P_later \approx \exp(-y * (1.5976 + 0.070566 * y^2)) \quad \text{if elapsed} > \mu$$

$$\phi = -\log_{10}(P_later)$$

The benefit is adaptivity: under jittery networks, μ and σ rise to absorb the jitter, so the same actual elapsed time produces a smaller ϕ . Under steady networks, μ and σ are tight and small absences accrue suspicion quickly.

A subtlety: the formula above is *Akka's* model, which assumes Normal inter-arrivals. *Cassandra* uses an Exponential model in its production implementation, with a different approximation. The same Φ_dead threshold means different things under the two models — at $\Phi_dead = 8$, an Akka-style detector fires significantly earlier than a Cassandra-style one. This paper retains 8 as the default and reports false-positive rates at $\Phi \in \{3, 5, 8, 12\}$ so the reader can see the trade.

3.3 Proposed formula (deliverable)

A worker is declared **DEAD** when $\phi(\text{now} - \text{last_heartbeat}) \geq \Phi_dead$, where ϕ is computed from the empirical distribution of inter-arrival times over a sliding window of the last N samples (Akka / Normal approximation), with bootstrap fallback to fixed-window thresholding while the window contains fewer than `min_samples` observations.

Defaults: $N = 1000$, `min_samples = 10`, $\Phi_missing = 1.0$, $\Phi_dead = 8.0$.

Implementation: `internal/detector/phi.go`.

3.4 Empirical comparison

The demo script (`scripts/run_demo.sh`) runs two Monitors side-by-side: one with Φ ($\Phi_dead = 8$), one with Fixed Window ($k_dead = 3$). Five workers run; `worker-3` self-terminates at $t=20s$, `worker-4` injects 2.5 s mean lag with 1 s stddev against a 1 s heartbeat interval. The observed outcome:

- Both monitors declare `worker-3` DEAD within ~ 10 s.
- The Fixed monitor declares `worker-4` DEAD spuriously at least once during the run.
- The Φ monitor never declares `worker-4` DEAD.

[image]

3.4.1 Φ_dead threshold sweep

§3.2 noted that Akka's polynomial gives different probability semantics than Cassandra's exponential, so a fixed threshold like $\Phi_dead = 8$ means materially different things under the two models. To validate that 8 is a defensible default for the Akka model used here, we ran a $\Phi_dead \in \{3, 5, 8, 12\}$ sweep against the same alive-but-jittery worker for 60 seconds in two scenarios: continuous **lag** (mean 2.5 s, stddev 1 s — every heartbeat arrives, just delayed) and **drop** (50% of heartbeats skipped — gaps but no lag). Script: `scripts/phi_sweep.sh`. Data: `phi_sweep.csv`.

Φ_{dead} scenario false-positive DEADs (60 s)

3	lag	0
5	lag	0
8	lag	0
12	lag	0
3	drop	1
5	drop	0
8	drop	0
12	drop	0

[image]

Two findings the paper relies on:

1. **Phi is robust to lag at any reasonable threshold.** Even at the most aggressive $\Phi_{\text{dead}} = 3$, no false positives appeared under continuous $2.5 \text{ s} \pm 1 \text{ s}$ lag. Phi's σ adapts to the jittery distribution: each delayed-but-arriving heartbeat is added to the sliding window, which inflates μ and σ symmetrically and keeps $(\text{elapsed} - \mu) / \sigma$ small enough that phi stays under 3.
2. **Drop scenarios are where Φ_{dead} matters.** With 50% of heartbeats skipped — no lag, just gaps — $\Phi_{\text{dead}} = 3$ produces one false positive in 60 s, while $\Phi_{\text{dead}} \in \{5, 8, 12\}$ produce none. This is the empirical justification for keeping the conservative default at 8: against gap-shaped failures, the marginal cost of waiting (a few hundred ms extra detection latency vs. $\Phi_{\text{dead}} = 5$) buys a meaningfully lower false-positive rate.

4. Implementation

The implementation is in Go and uses gRPC (google.golang.org/grpc) for transport. Module layout:

- `internal/detector/` — failure detector interface and the two implementations
- `internal/monitor/` — gRPC server, registry, evaluator, pull poller, TUI
- `internal/worker/` — heartbeat pusher, ping responder, chaos controller
- `cmd/{monitor,worker}/` — process entry points

The detector's interface is transport-agnostic: it sees only (`workerID`, `arrivalTime`) events, never knowing whether they came from a push stream or a pull reply. This is the cleanest design choice in the system and is what makes the empirical comparison honest.

5. Results

(Charts above.) The headline numbers, as observed on a single MacBook Pro (M-series, macOS):

- Detection latency at $N = 1000$: push/Phi = 1186 ms, push/Fixed = 9910 ms, pull/Phi = 642 ms, pull/Fixed = 9833 ms. Phi fires roughly an order of magnitude earlier than Fixed because tight σ on steady arrivals lets phi exceed Φ_{dead} well before $k_{\text{dead}} \times \text{hb_interval}$ elapses.
- Pull/Phi being faster than push/Phi at $N = 1000$ is not a contradiction of "push wins":

pull's Monitor-driven cadence has tighter σ (the Monitor controls the timer; jitter is dominated by goroutine scheduling, not OS pipe scheduling), which lets phi cross Φ_{dead} in fewer poll-intervals after the gap. Push σ inherits Worker-side ticker drift on top, which inflates μ and σ slightly. The push/pull RSS gap (78 MB vs 200 MB at $N=1000$) and the firewall argument are unaffected.

- Monitor RSS at $N = 1000$ is materially higher under pull (196 MB) than push (76 MB); pull holds one outbound `*grpc.ClientConn` per worker plus per-poll context state, while push reuses one inbound stream per worker.
- Phi never declares the jittery-but-alive worker dead; Fixed Window with `k_dead = 3` declares it dead within 30 seconds in every run (see `demo-fixed.jsonl`).

Raw bench data: `bench.csv` (12 rows, $N \in \{10, 100, 1000\}$, $\text{mode} \in \{\text{push}, \text{pull}\}$, $\text{detector} \in \{\text{phi}, \text{fixed}\}$).

6. Conclusion

For a single-monitor / N -worker dead man's switch, the recommendation is **push transport with the Phi Accrual detector**. Push is firewall-friendly and slightly cheaper on the Monitor; Phi is adaptive to jitter and produces materially fewer false positives under realistic network conditions. The implementation tops out at ~ 1000 workers per monitor on commodity hardware before the gRPC server's per-stream cost dominates; multi-monitor / gossip-based detection (SWIM, Memberlist) is the natural next step but is out of scope for this paper.

References

1. Hayashibara, N.; Défago, X.; Yared, R.; Katayama, T. *The ϕ Accrual Failure Detector*. Proceedings of the 23rd Symposium on Reliable Distributed Systems (SRDS 2004), pages 66–78.
2. Lakshman, A.; Malik, P. *Cassandra: A Decentralized Structured Storage System*. SIGOPS Operating Systems Review 44(2), April 2010, pages 35–40.
3. Akka source. `akka.remote.PhiAccrualFailureDetector` — <https://github.com/akka/akka>
4. Cassandra source. `org.apache.cassandra.gms.FailureDetector` — <https://github.com/apache/cassandra>